Todo app: product and technical requirements

Draft for implementation review | Learning project | 5 November 2026

Executive summary

Build an offline-first todo app to learn Go, React, Wails, persistence, and cloud collaboration. Users manage personal and project tasks without a connection, then optionally sign in to sync and share projects.

The full target is Windows, macOS, Linux, Android, and iOS. Deliver desktop first, cloud features second, and mobile third. Wails v3.0.0 is the proposed pinned baseline: desktop is stable; Android and iOS are experimental [1, 2]. An iOS build needs macOS and full Xcode.

Scope and success

- Create, edit, complete, reopen, and delete tasks. Keep global tasks private and outside projects. Support one subtask level.
- Use Firebase Authentication for Google and email/password. Sync signed-in data, invite project members by email, and revoke individual device sessions.
- Prove local durability by restarting offline after changes. Prove sync convergence on two devices, conflict recovery without lost edits, and denial of revoked-device requests.
- Demonstrate owner/editor/viewer authorization through positive and negative API tests. No revenue, adoption, or performance results are claimed.

Agreed product rules

Title is required. Description and tags may be empty. Due date is optional, stored as a calendar date without time or timezone conversion. A task belongs to exactly one personal/project scope. A subtask inherits its parent's scope and cannot contain subtasks. Parent and subtask completion are independent.

Project sharing uses owner, editor, and viewer roles. A viewer cannot mutate tasks. Owners manage members and delete projects. Personal tasks are never shared. Resolve concurrent edits by asking the user to keep the server version or submit a revised local version.

Exclusions

No recurring tasks, reminders, attachments, comments, task assignment, web edition, background daemon, or app-store launch requirement. Sync runs while the app is active and on resume; mobile background execution is not guaranteed.

Learning milestones

- M1: Local desktop app, SQLite migrations, tasks/projects/subtasks, validation and tests.
- M2: Firebase sign-in, Go sync API, explicit conflicts, invitations and roles, device sessions.
- M3: Android and iOS builds, responsive layouts, secure storage, lifecycle and OAuth checks.

User stories: local work

US-01: Work offline

As a local user, I want my tasks saved on this device so I can work without an account or connection. Design: WF-01 and WF-03.

- All local task actions work on a fresh launch with networking disabled.
- A successful save commits the task and queued operation in one SQLite transaction; the UI reports success only after commit.
- Restarting preserves saved changes. A disk error leaves the previous state intact and shows a retry message.
- Local-only mode sends no task data to the cloud. First sign-in asks before importing local data.

US-02: Manage task details

As a user, I want a title, description, due date, and tags so I can understand and organize a task. Design: WF-01 and WF-03.

- Reject a blank title after trimming; allow empty description/tags and a cleared due date.
- Create, edit, complete, reopen, and delete work offline. Due dates remain the same calendar date on every platform.
- Show validation beside the field and keep typed input after a failed save.
- Proposed limits: title 200 characters, description 10,000, up to 20 tags of 30 characters each. Trim and deduplicate tags case-insensitively.

US-03: Organize tasks and subtasks

As a user, I want global tasks, projects, and subtasks so I can separate standalone work from related work. Design: WF-01 and WF-03.

- Create and rename projects; display global tasks separately from project tasks.
- Add subtasks to a root task only. Reject cycles, self-parenting, and a second nesting level in both Go and the cloud API.
- Subtasks inherit scope and permissions. Completing a parent does not change its subtasks.
- Proposed move rule: move a root and all its children together; require destination write access. Detaching a subtask makes it a root in the same scope.

US-04: Delete safely

As a user, I want clear deletion behavior so I do not remove related work by accident. Design: WF-01.

- Confirm deletion of a parent with children or a project; show how many tasks are affected.
- Proposed rule: deleting a parent deletes its children; deleting a project deletes its tasks. Only its owner can delete a shared project.
- Retain synced deletion tombstones; never resurrect a deleted task from a stale device automatically.
- Treat a concurrent delete/edit as a conflict. Offer to copy local content into a new task only where the user still has write access.

User stories: accounts and collaboration

US-05: Sign in and retain local work

As a user, I want Google or email/password sign-in so I can use the same account on different devices. Design: WF-04.

- Support registration, sign-in, email verification, password reset, cancellation, and sign-out. Require verified email for cloud collaboration.
- Google sign-in uses a supported system-browser/native flow, not an embedded Google login page [4].
- After first sign-in, offer Import local data or Keep local only. Import is idempotent and preserves a local copy until acknowledged.
- Partition data per account. Signing out or switching accounts never exposes another account's cached data; warn about unsynced changes before clearing credentials.

US-06: Sync and resolve conflicts

As a signed-in user, I want devices to converge without silently losing edits. Design: WF-01 and WF-02.

- Queue offline writes durably, retry transient failures, and expose Local only, Offline, Pending, Synced, Conflict, or Sign-in required.
- Replaying a mutation ID never applies it twice. Apply fetched changes and the sync cursor in one local transaction.
- A stale entity version returns a conflict with both versions. Keep the local draft until the user resolves it; compare again before accepting a replacement.
- Revoked membership or sessions stop uploads. Preserve rejected edits privately; never bypass permission checks to replay them.

US-07: Invite and manage project members

As an owner, I want to invite another email as editor or viewer so we can share a project. Design: WF-02.

- Create an online-only invitation; access begins only after acceptance by the matching verified email. Existing and new accounts can accept.
- Proposed invitations expire after seven days and can be cancelled or declined. An invalid, expired, or cancelled link grants no access.
- Editors manage tasks but not membership, ownership, or project deletion. Viewers can read only. Enforce this on every API request.
- Owner can change a member's role or remove them. Removal stops future cloud access; cached copies cannot be remotely erased while offline.

US-08: Manage device sessions

As a signed-in user, I want to list and revoke one device so I can stop its cloud access without signing out others. Design: WF-05.

- Show device name, platform, last server contact, and current-device indicator. Last contact is not live online status.
- Require online reauthentication and confirmation before revocation. Reject subsequent requests for that session after the server records revocation.
- Other sessions remain valid. On next contact, the revoked client clears credentials and requests sign-in; existing offline data cannot be remotely wiped.
- Revoked credentials alone cannot register a replacement session: enrollment needs a new, single-use proof of fresh interactive sign-in.

Technical specifications

Components and data flow

Proposed architecture: React + TypeScript renders the UI; Wails v3 exposes a thin Go service bridge. Go owns validation, SQLite, migrations, the durable outbox, sync, and credential access. Keep task logic independent of Wails for unit testing. Pin the Go toolchain and all dependencies when implementation starts; verify current platform requirements.

Cloud path: local Go sync client -> HTTPS Go API -> Cloud Firestore. Firebase Authentication supplies identity. The API verifies identity and device session, then enforces ownership and project roles. Clients do not read or write application data directly in Firestore. Deny direct client access in Firestore Rules; the server Admin SDK bypasses those rules, so API authorization is mandatory.

Run the Go API on a managed container service, proposed Cloud Run. Use managed service identity, not a service-account key shipped in the app. Firestore is a proposed storage choice, not a confirmed user requirement. No cloud resources are provisioned by this document.

Minimal data model

- Task: id, ownerUid, projectId nullable, parentId nullable, title, description, dueDate nullable (YYYY-MM-DD), tags[], completed, version, createdAt, updatedAt, deletedAt nullable.
- Project: id, ownerUid, name, version, deletedAt. Membership: projectId + uid, role (owner/editor/viewer). One owner per project; ownership transfer is out of scope.
- Invitation: id, projectId, normalizedEmail, role, tokenHash, expiresAt, status. Acceptance verifies account email and creates membership transactionally.
- Device session: sessionId, uid, deviceName, platform, tokenHash, createdAt, lastSeenAt, revokedAt. Never use a caller-supplied device ID as proof of possession.
- Local outbox: mutationId, accountPartition, entityId, baseVersion, operation, payload, state. Cloud change feed: per-user sequence, entityId, version, deletion/access-removal marker.

Proposed API contract

- POST /sessions/enroll: exchange fresh sign-in proof for an opaque device credential. GET /devices and POST /devices/{id}/revoke: current user's sessions only.
- POST /sync/push: bounded batch of mutation IDs and base versions; return accepted, conflict, or denied per mutation. GET /sync/pull?cursor=...: authorized change pages and a new cursor.
- POST /projects/{id}/invitations; POST /invitations/{token}/accept; PATCH or DELETE /projects/{id}/members/{uid}. Check owner role and email verification server-side.

Use 401 for identity/session failures, 403 for permission denial, 409 for conflicts, and 429 for throttling. Reject unknown fields and oversize payloads. A failed item must not hide the status of other items in a batch.

Sync, identity, and security rules

Local-first sync algorithm

1. Validate a change in Go, then commit the entity and outbox entry atomically. An unsent create followed by edits can be compacted; do not send a version-dependent mutation before its predecessor is acknowledged.
2. On startup/resume, reconnect, manual retry, and a proposed 15-second foreground interval, check the session and pull changes. Never overwrite a pending local draft or its base version: stage incoming remote versions separately for comparison. No connection leaves the queue intact.
3. Push each eligible mutation with a unique ID and expected version. The API checks current membership, compares versions, and transactionally records the entity, idempotency result, and change-feed entries.
4. Pull authorized changes after pushing. Commit each page with its cursor. A crash before commit repeats the page safely. Revisions use server ordering, not device clocks.
5. Retry timeouts, 429, and 5xx with capped exponential backoff and jitter. Do not auto-retry validation/permission failures. A 409 pauses that entity until the user chooses a version.

Proposed learning-project simplification: retain mutation receipts and tombstones without automatic expiry. Enforce dataset limits to control costs. If a future retention policy drops a cursor, require a full authorized snapshot and rebase pending work; never silently skip missing changes. Fan out project changes to each member's feed; granting access seeds a snapshot, and removal emits an access-removal marker. Recheck permissions when reading the feed so an old entry cannot leak removed-project data.

Device-specific authorization

Firebase's refresh-token revocation is user-wide [3]. Use an additional opaque device credential, hashed at rest on the server, on every application-data request. Check its revocation state without a permissive cache. After revocation commits, all newly authorized requests for that session fail; requests already authorized may finish.

Proposed enrollment: begin a short-lived, one-use server challenge before interactive Firebase sign-in. Validate a fresh Firebase auth_time, identity, and unused challenge server-side, then issue the device credential once. Refreshing an old Firebase token must not satisfy fresh sign-in. Recent reauthentication authorizes device revocation. Prove these flows in a security spike before implementing sharing.

Protect credentials and account boundaries

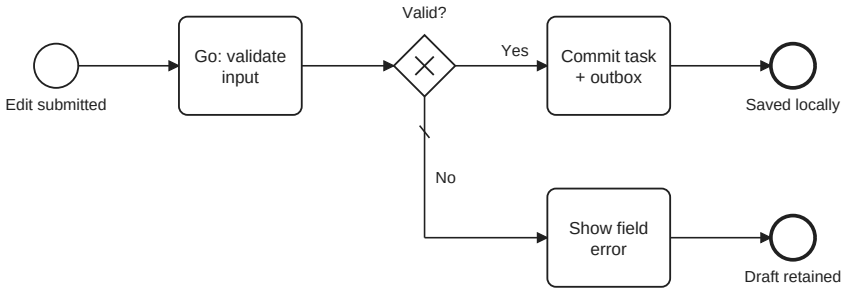
Store refresh/device credentials in platform secure storage accessed through Go/native adapters; do not put them in frontend localStorage or ordinary SQLite. Keep short-lived identity tokens in memory. Redact credentials, descriptions, and emails from logs. Use HTTPS, bound request sizes, rate limits, and dependency pinning. SQLite encryption at rest is not promised; protect app files with OS permissions and disclose that local tasks are readable to someone with device access.

For desktop Google sign-in, validate state, nonce, PKCE, and the redirect through a supported browser flow, then exchange the Google credential with Firebase. Mobile uses supported native/browser SDK adapters rather than desktop loopback assumptions [4, 5]. Email/password is sent only to Firebase over TLS; do not store passwords. Sign-out keeps account data partitioned and asks before discarding pending work.

BPMN: save locally and sync

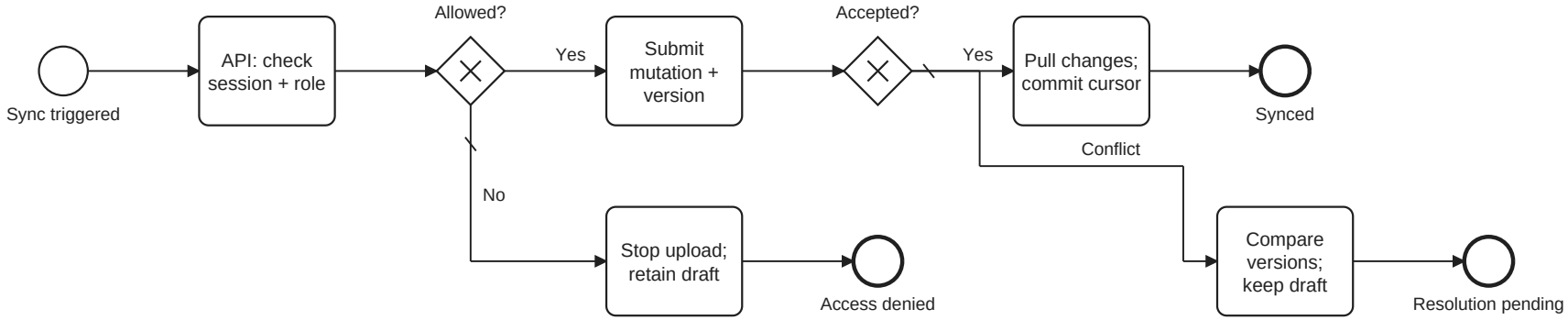
Circles are events; rounded boxes are activities; diamonds are exclusive decisions. A thick circle ends a path. Slashed branches are defaults. These are non-executable BPMN 2.0 models.

Local save | US-01 and US-02



A database failure reports an error instead of Saved locally; the transaction rolls back and the draft stays available.

Online mutation cycle | US-06

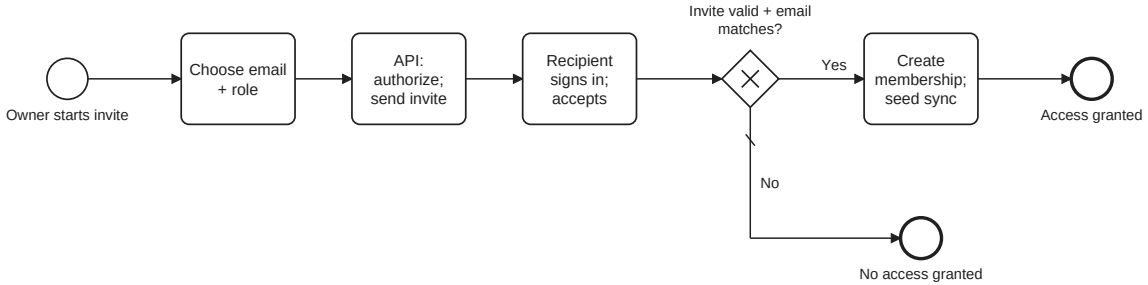


Network failures retain the queue and retry later. The conflict branch covers version conflicts after validation; other validation errors retain the draft. A resolution submits a new version-checked mutation. The API rechecks authorization when committing.

BPMN: sharing and device access

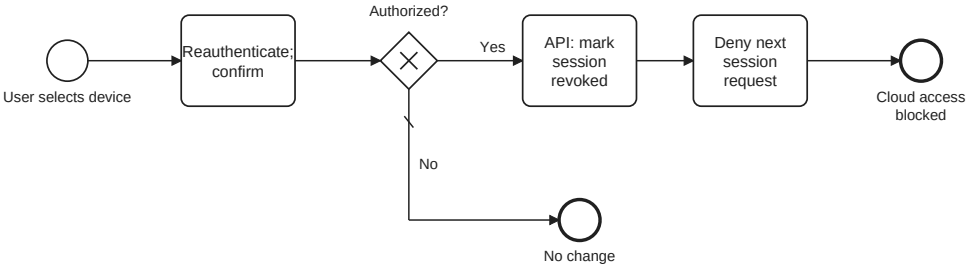
Each diagram is one end-to-end coordination process. Activity labels identify the owner, recipient, or API; sequence flows stay inside that process. These are documentation models, not engine deployment files.

Project invitation | US-07



The API checks the owner before sending. Acceptance checks expiry, cancellation, project existence, verified email, and role. An ignored invitation expires without granting access; decline cancels acceptance. Delivery errors leave a retryable pending invitation.

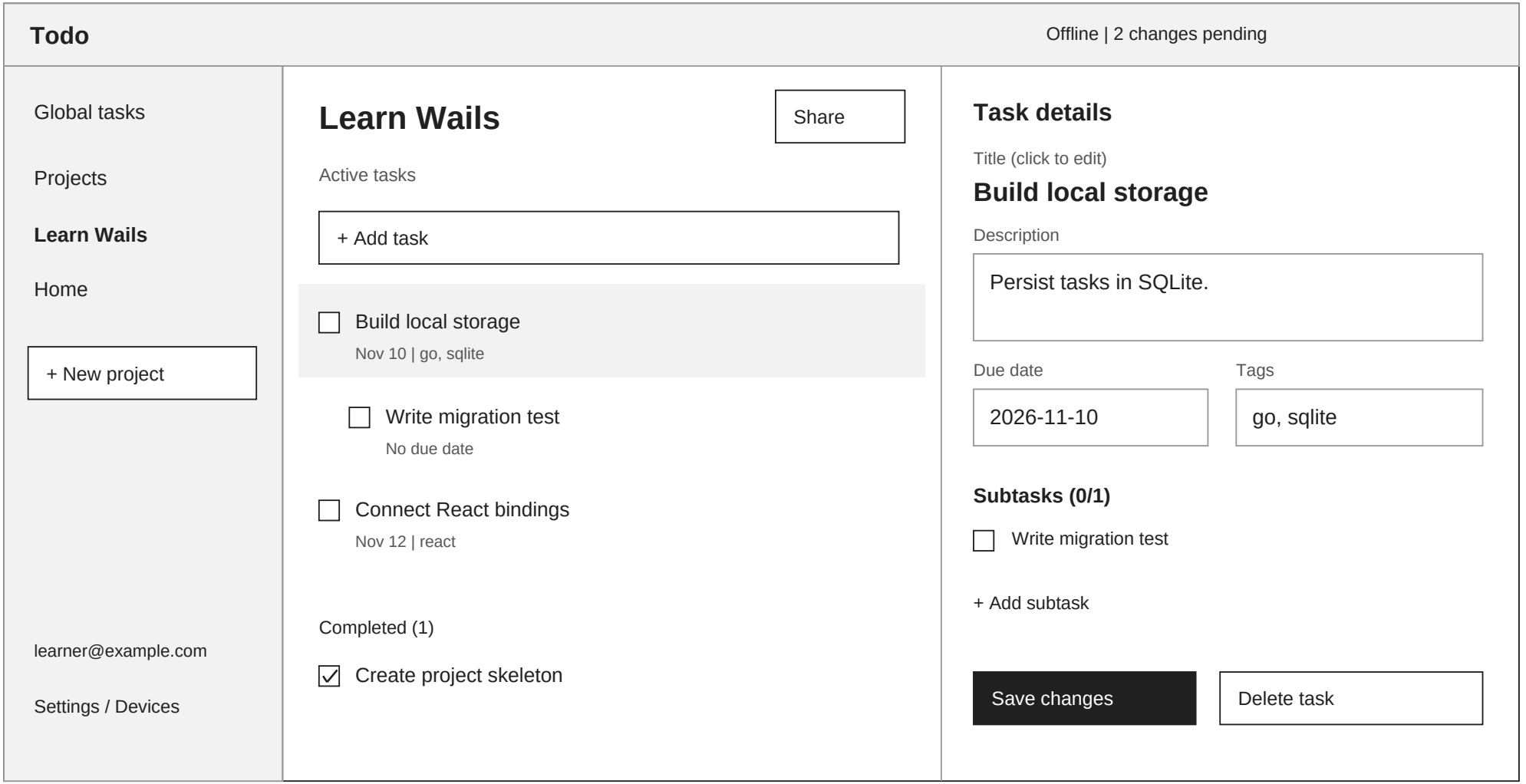
Individual device revocation | US-08



Authorization includes current-session ownership and fresh reauthentication. Next request means the next request authorized after the revocation commit; already-authorized work may finish. No network means no remote wipe or immediate local logout.

Desktop wireframes

WF-01: sidebar, task list, and details. Fictional sample data; static grayscale sketches, not a working app.



Viewer: hide editing controls and show Read-only. Empty list: No tasks yet + Add task. Long titles wrap without hiding the task actions. Saved locally and Synced remain distinct states.

Sharing, conflicts, and devices

WF-02: share and resolve conflicts

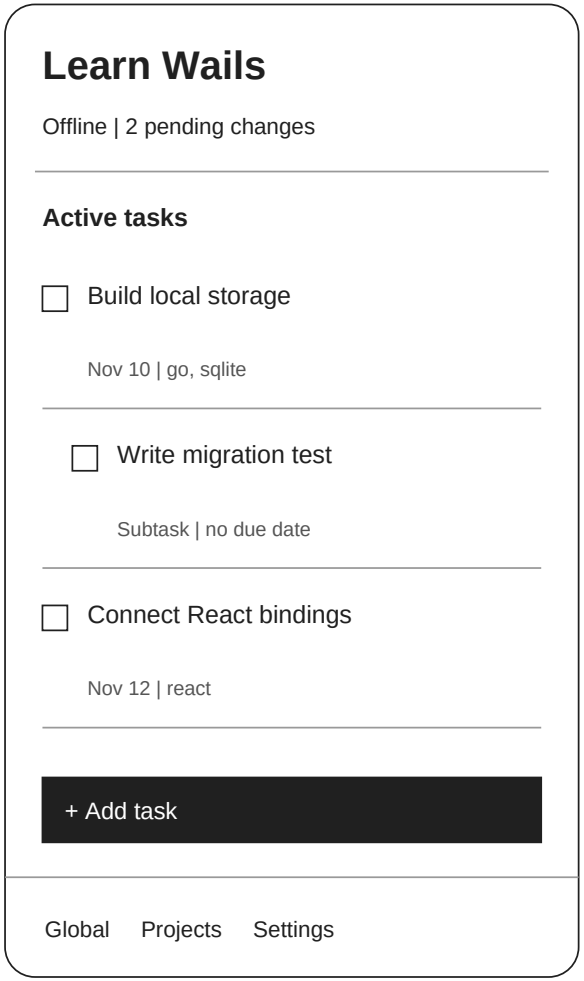
Invites need a connection. Keep conflict drafts until confirmation; compare again if the server version changes.

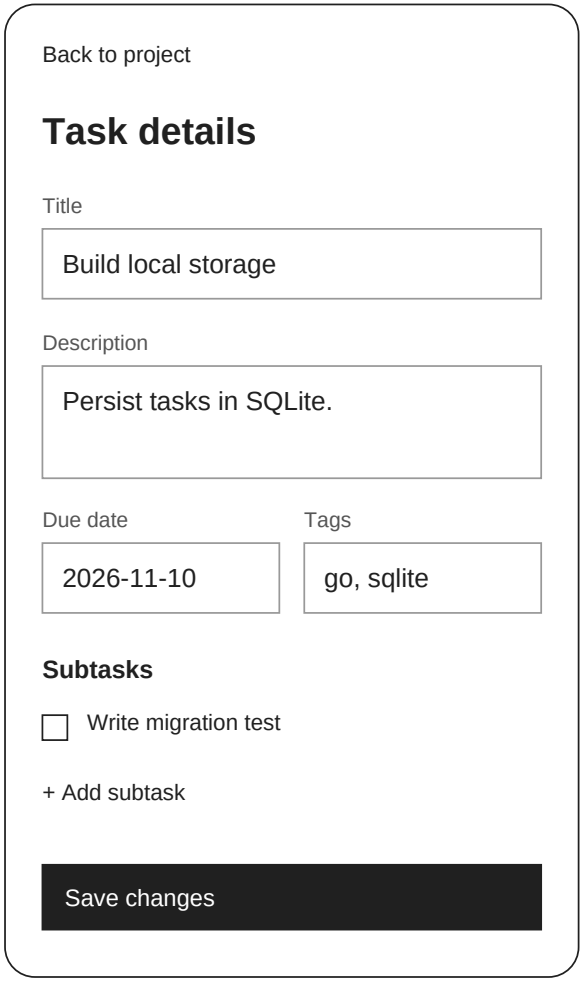
Device management | WF-05

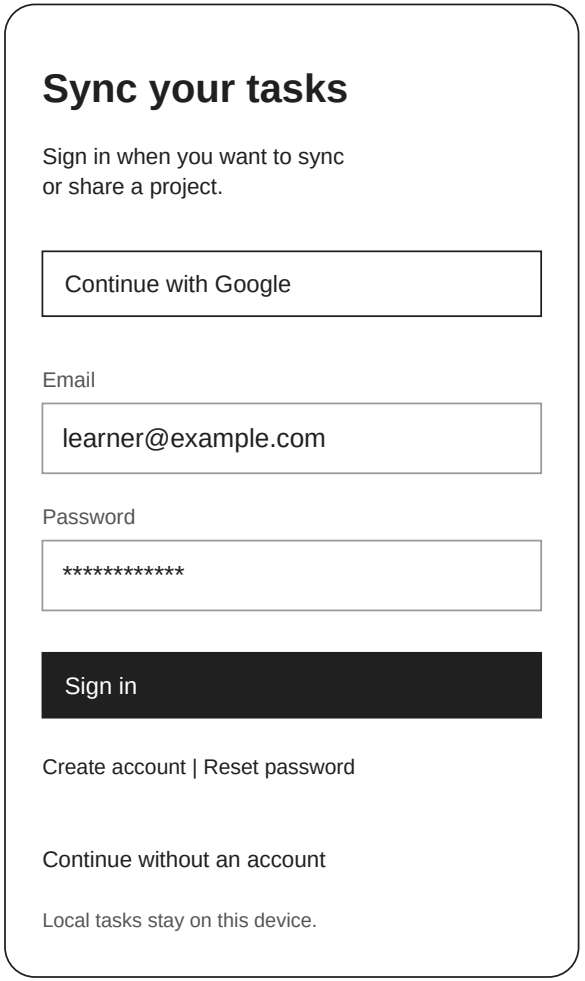
Require online reauthentication and a server acknowledgement. Revocation stops cloud access on the next authorized request; it cannot erase downloaded data. Show Retry if offline.

Mobile wireframes

WF-03: list opens task details. WF-04: optional sign-in. Shared React UI adapts to native safe areas and the on-screen keyboard. Fictional sample content.







After sign-in, ask whether to import local data. Invalid credentials: keep the email and show a retry/reset action. Unverified email: offer resend verification. Revoked session: pause sync and require sign-in. Touch targets are at least 44 CSS px in the built app; these PDF sketches are scaled.

Risks, verification, and remaining decisions

Release checks

- Go unit/race tests: validation, one-level hierarchy, account partitions, independent completion, transactions, and migration rollback/recovery behavior.
- Sync integration tests: two devices, offline restart, lost responses, repeated IDs, ordered mutations, partial batches, edit/edit and delete/edit conflicts, and interrupted cursor commits.
- Authorization tests: viewer writes denied; editor membership changes denied; invitation email mismatch denied; removed members denied; direct Firestore reads/writes denied.
- Device tests: revoke A while B stays valid; A's stale Firebase tokens cannot enroll again; in-flight request boundary is documented; offline data is not falsely described as erased.
- UI/manual checks: keyboard-only use, visible focus, screen-reader labels, non-color sync states, long titles, empty lists, save errors, safe areas, and mobile keyboard overlap. Native runs are required on all targets.

Proposed budgets, not measured results: local saves within 100 ms at the 95th percentile on a stated test device with 5,000 tasks; online changes visible on a second active device within 30 seconds under normal test-network conditions. Measure before treating these as release gates.

Risks and mitigations

Mobile Wails support is experimental: build a small Android/iOS proof before promising parity. Google OAuth and secure storage require platform-specific work: validate one successful flow on each OS. Individual device revocation adds a server authorization layer: test it before collaboration. Email delivery and Firestore usage can incur cost: use test accounts, budget alerts, and bounded datasets.

Proposed defaults still needing review

Approve or revise TypeScript, SQLite, Cloud Run/Firestore, input limits, cascade deletion, move behavior, invitation expiry, and the 15-second sync interval before implementation. Also decide an app name, minimum OS versions, supported Go/React versions, invitation email provider, and packaging/signing approach. No deadline or cloud budget has been agreed.

Sources checked on 5 November 2026

[1] Wails release and platform status: https://wails.io/

[2] Wails mobile requirements: https://wails.io/guides/mobile/

[3] Firebase session revocation: https://firebase.google.com/docs/auth/admin/manage-sessions

[4] Google native OAuth and embedded-user-agent restrictions: https://developers.google.com/identity/protocols/oauth2/native-app

[5] Firebase sign-in using Google credentials: https://firebase.google.com/docs/auth/web/google-signin

The BPMN files describe behavior, not an executable Camunda deployment. Wireframes are static planning sketches with fictional content, not screenshots of a working app.